

Лабораторная работа № 5 ФАЙЛОВОЕ ХРАНИЛИЩЕ

Задание:

Необходимо реализовать удаленное файловое хранилище, предоставляющее REST API по протоколу HTTP. При реализации разрешается использовать веб-фреймворк, реализующий HTTP-роутинг и работу с HTTP-вызовами. Поддерживаемые функции:

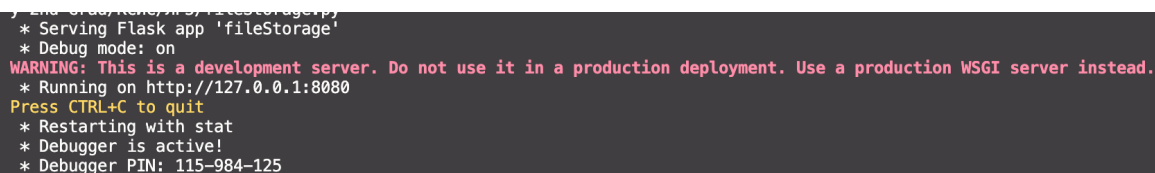
- загрузка файла в хранилище с помощью метода PUT;
- получение файла из хранилища с помощью метода GET;
- получение списка файлов каталога с помощью метода GET (в формате JSON);
- получение информации о файле (размер в байтах и дата последнего изменения) в виде HTTP-заголовков с помощью метода HEAD;
- удаление файла из хранилища с помощью метода DELETE.

Получение файла или содержимого каталога также должно быть возможным через браузер. Для тестирования использовала Postman.

Ход работы:

Для реализации данной задачи был выбран язык программирования Python и микрофреймворк Flask. Он обеспечивает удобный способ создания веб-сервисов с минимальным количеством шаблонного кода и берет на себя маршрутизацию запросов. Готовое решение представляет собой локальный сервер.

- 1) В самом начале подключаем библиотеки (Flask, os, datetime) и инициализируем переменные для сборки приложения. Запустим созданный код в терминале.



```
* Serving Flask app 'fileStorage'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8080
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 115-984-125
```

Рисунок 1 – Запуск сервера в терминале

- 2) Затем создадим методы для обработки каждого из запросов. Отправим файл в хранилище. Реализуем *метод PUT* для загрузки файла. В самом методе реализуем перекачку данных напрямую — сервер принимает байты и сохраняет их на диск. В Postman указываем адрес и тело запроса, которое будет содержимым файла.

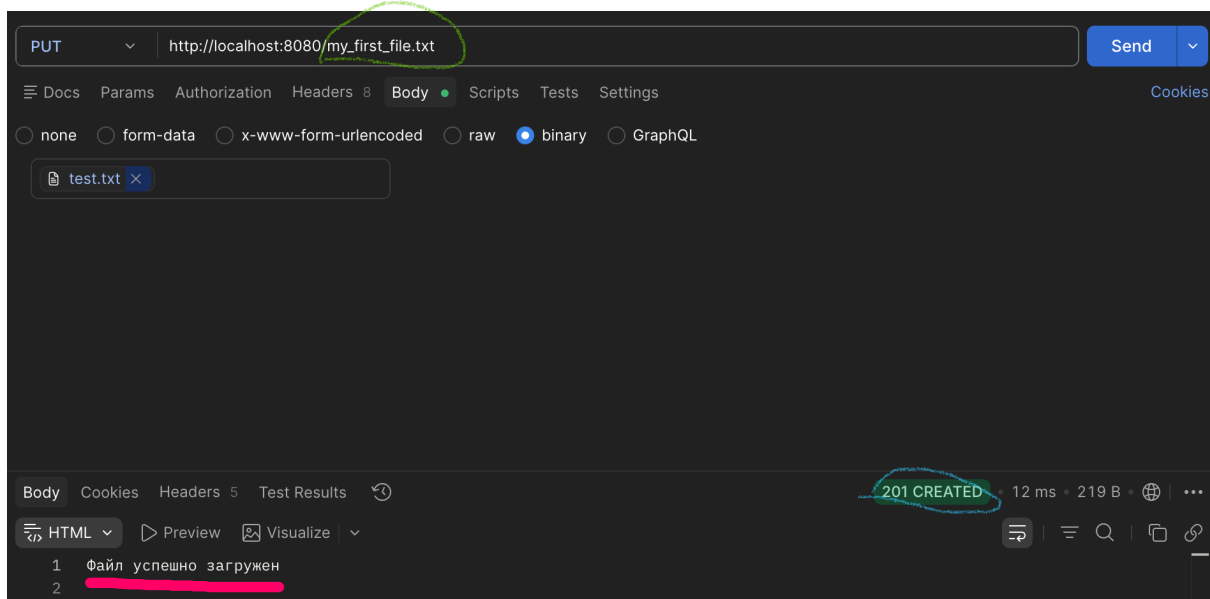


Рисунок 2 – Создание запроса PUT



Рисунок 3 – Что видим в терминале

Source	Destination	Protocol	Length	Info
127.0.0.1	127.0.0.1	HTTP	319	PUT /my_first_file.txt HTTP/1.1
127.0.0.1	127.0.0.1	HTTP	97	HTTP/1.1 201 CREATED (text/html)

Рисунок 3 – Отправка в хранилище (Wireshark)

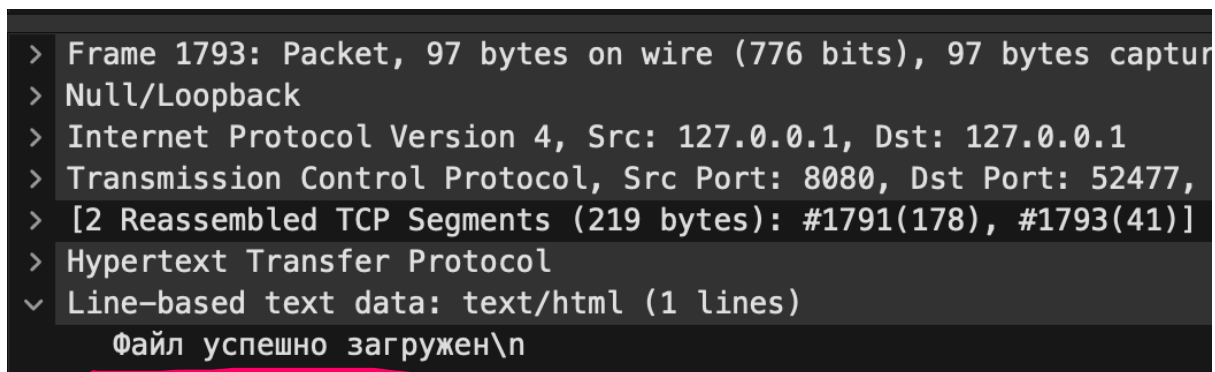


Рисунок 4 – Тело запроса PUT

Для отладки программы использую интерфейс loopback. В Wireshark начнём захват пакетов, предварительно включив фильтр отображения `http && tcp.port == 8080`.

- 3) Начнём с GET для просмотра информации и скачивания. Если запрошенный путь — это папка, то превращаем её содержимое в список и отправляем как JSON. Если это файл, то считываем его и отправляем клиенту. Для этого создадим GET-запрос в Postman

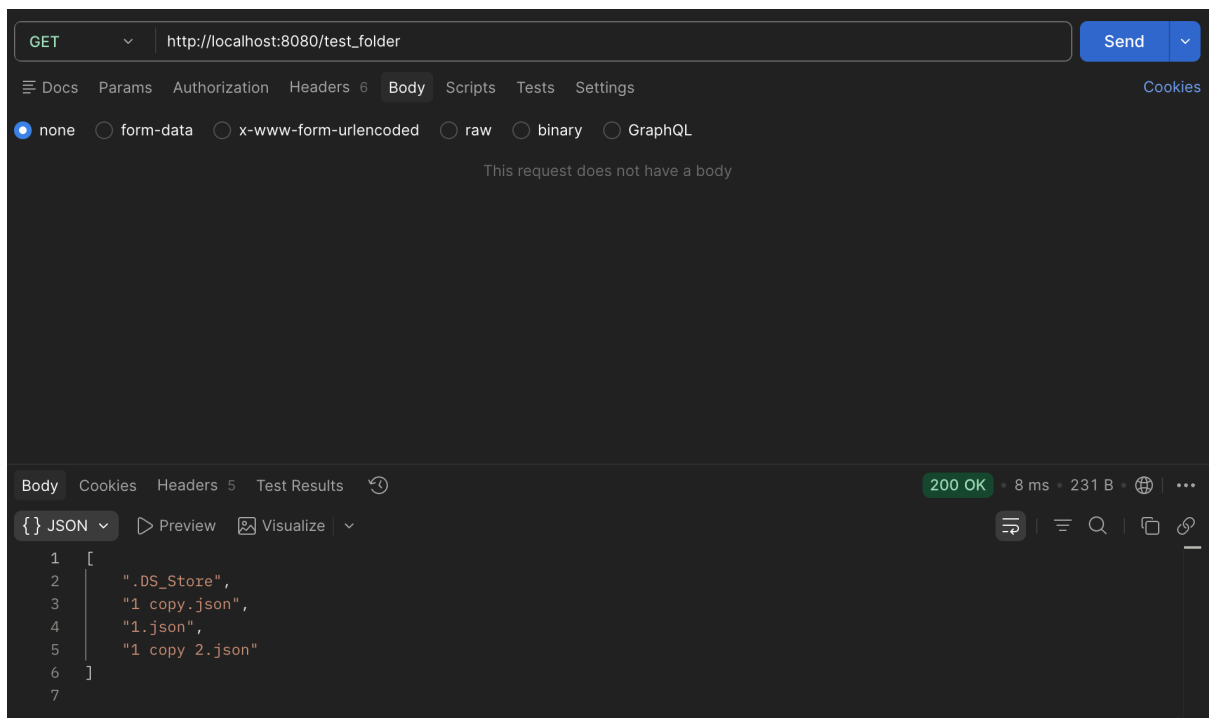


Рисунок 5 – Получение списка файлов каталога (Postman)

HTTP	268	GET /test_folder	HTTP/1.1
HTTP/JSON	122	HTTP/1.1 200 OK , JSON (application/json)	

Рисунок 6 – Перехват GET-запроса и ответ сервера в Wireshark

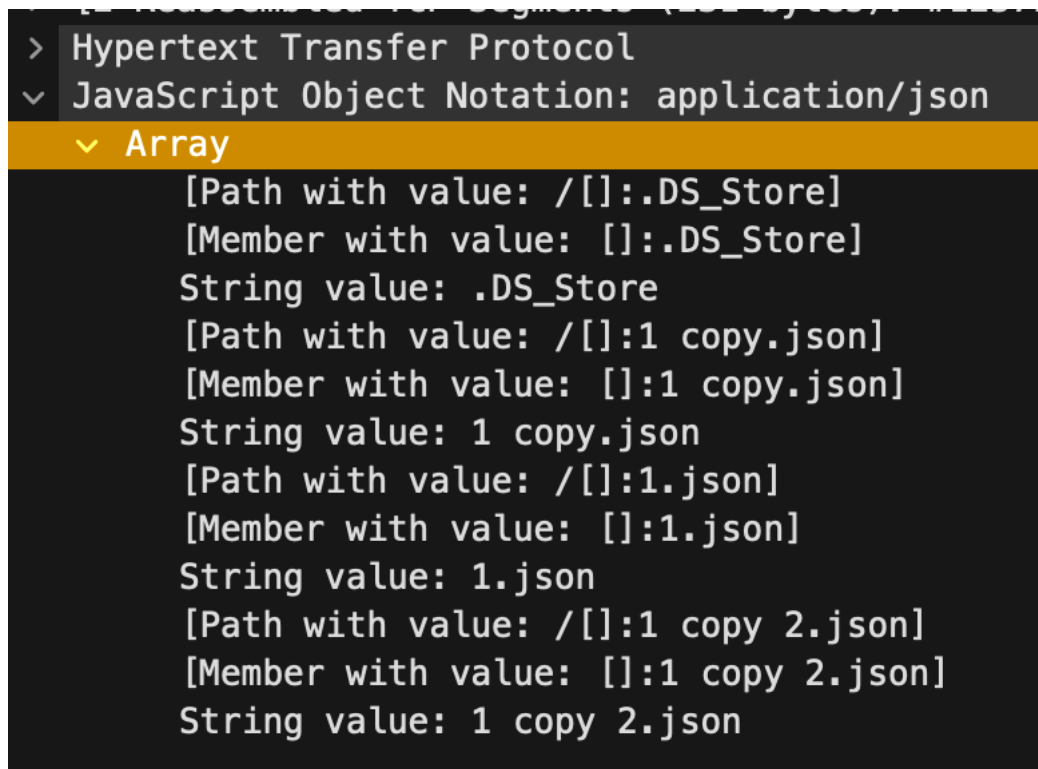


Рисунок 7 – Тело файла GET

```
127.0.0.1 - - [26/Apr/2026 16:22:49] "GET /test_folder HTTP/1.1" 200 -
```

Рисунок 8 – Результат в терминале

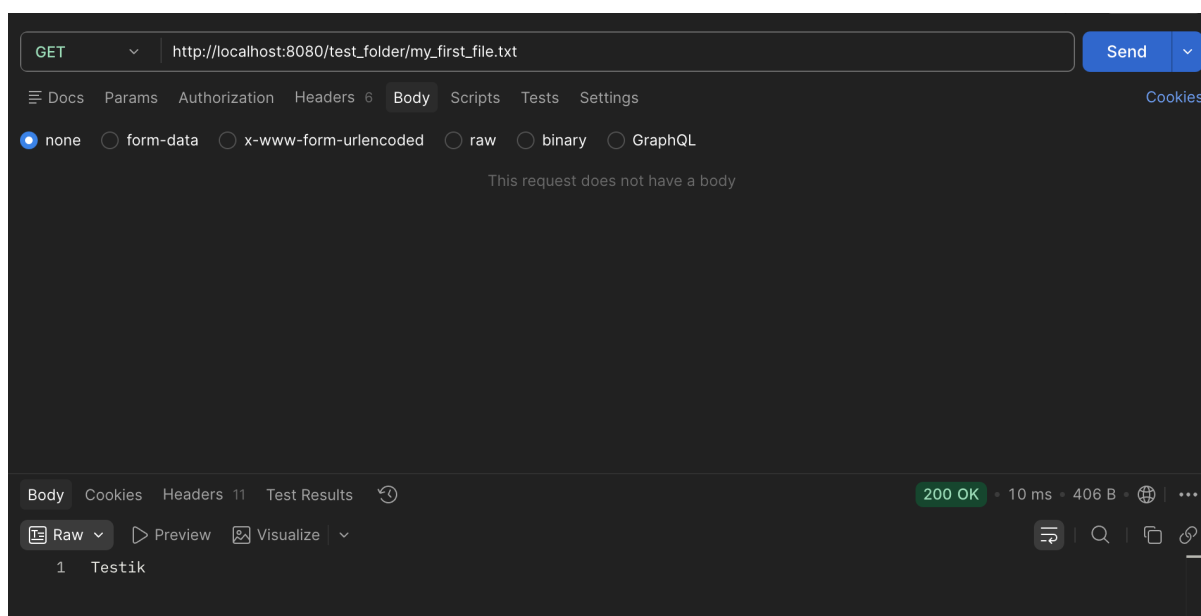


Рисунок 9– Скачивание файла через метод GET (Postman)

```
HTTP      286 GET /test_folder/my_first_file.txt HTTP/1.1
HTTP      62 HTTP/1.1 200 OK (text/plain)
```

Рисунок 10 – Показ скачивания в Wireshark

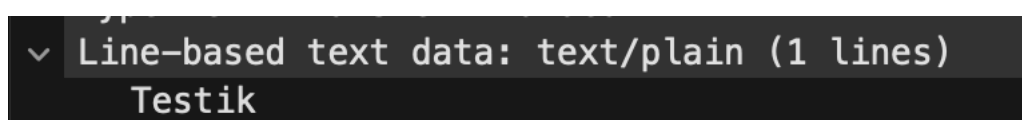


Рисунок 11 – Тело отправленного файла GET

```
127.0.0.1 - - [26/Apr/2026 16:24:00] "GET /test_folder/my_first_file.txt HTTP/1.1" 200 -
```

Рисунок 12 – Результат в терминале

- 4) Продолжим с методом HEAD. Здесь отправляем только самое необходимое: размер, дату и время последнего изменения. Эти данные передаются в виде HTTP-заголовков, без тела ответа.

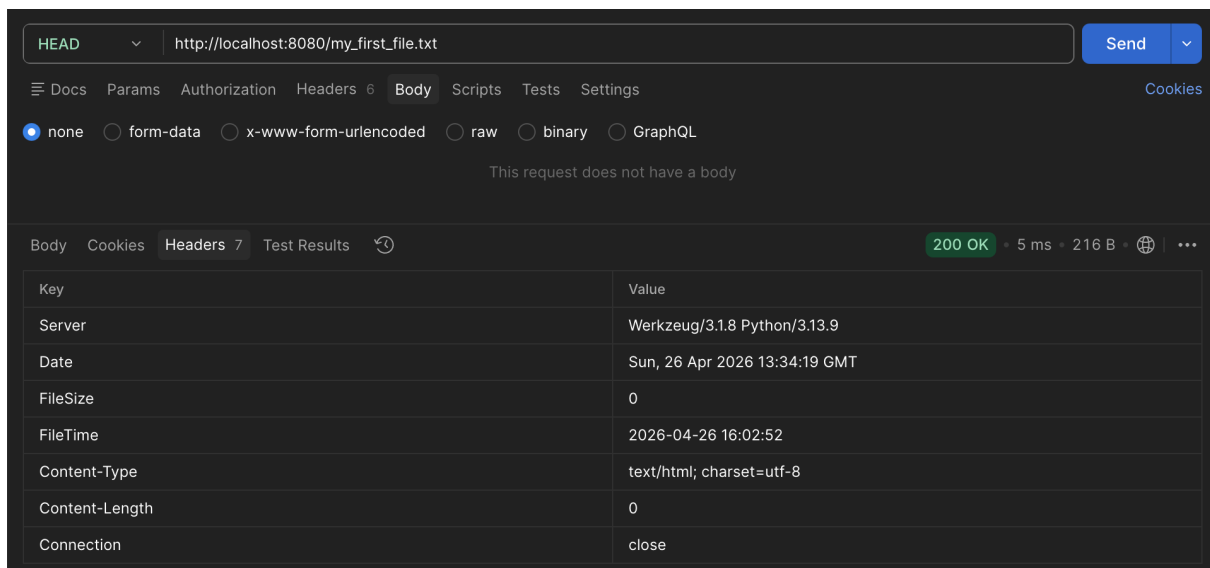


Рисунок 13 – Получение информации о файле HEAD (Postman, вкладка Headers)

```
HTTP      275 HEAD /my_first_file.txt HTTP/1.1
HTTP      272 HTTP/1.1 200 OK
```

Рисунок 14 – Перехват HEAD-запроса в Wireshark (виден обмен заголовками без передачи самого файла)

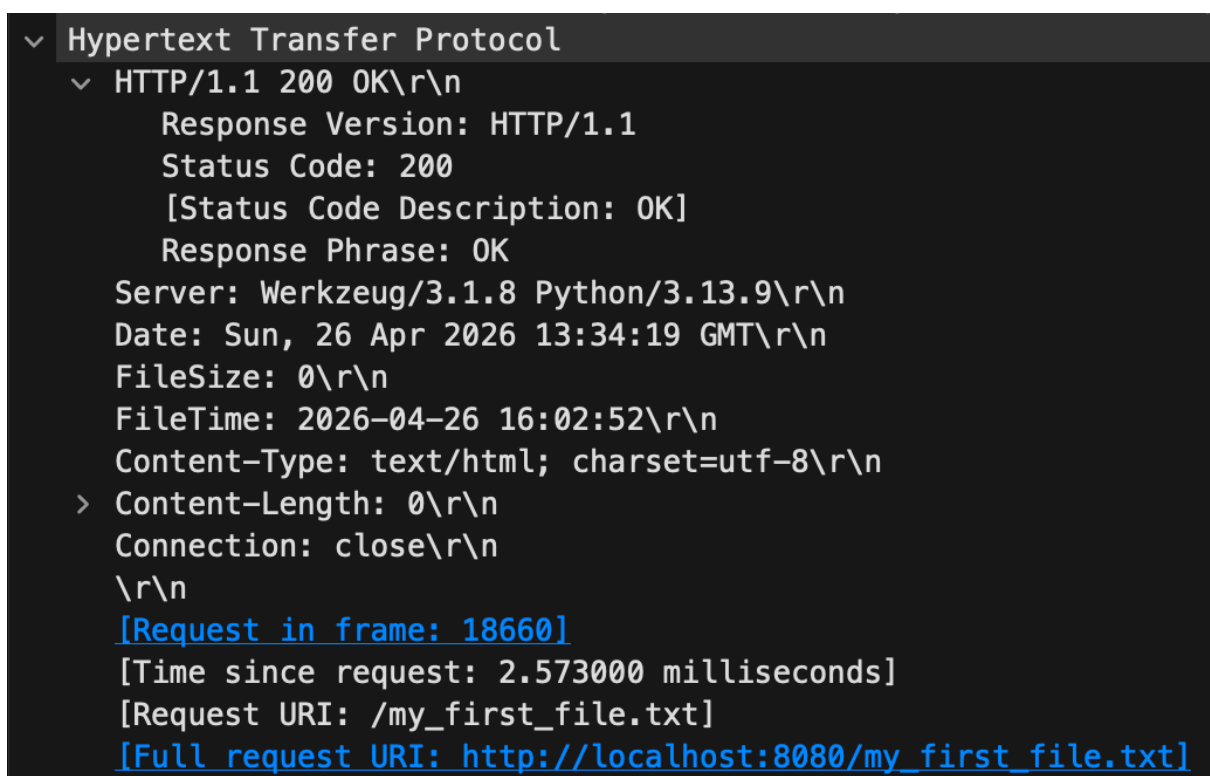


Рисунок 15 – Тело HEAD-ответа в Wireshark

```
127.0.0.1 -- [26/Apr/2026 16:34:19] "HEAD /my_first_file.txt HTTP/1.1" 200 -
```

Рисунок 16 – Результат в терминале

5) Теперь про DELETE. Через встроенные методы операционной системы (os.remove) создаём возможность удаления файла.

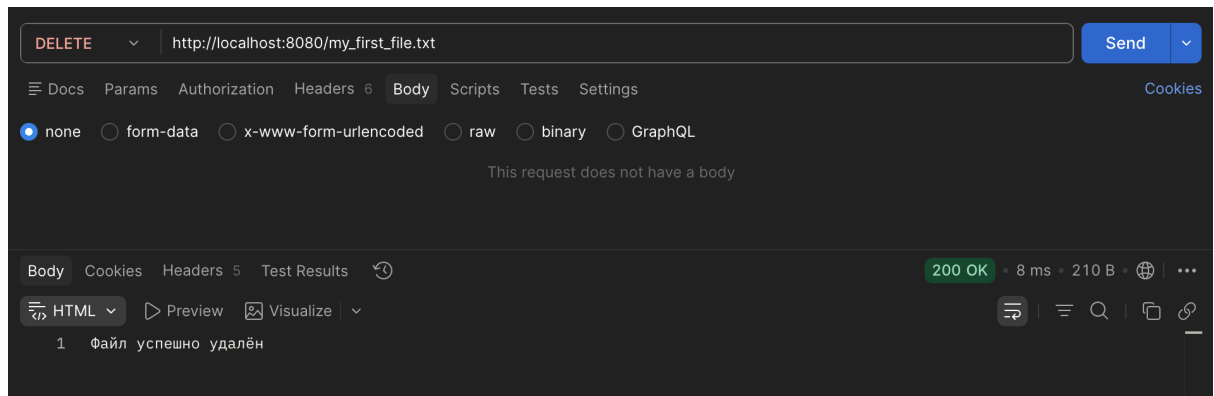


Рисунок 17 – Удаление файла через метод DELETE (Postman)

```
HTTP      277 DELETE /my_first_file.txt HTTP/1.1
HTTP      93 HTTP/1.1 200 OK (text/html)
```

Рисунок 18 – Перехват DELETE-запроса и ответа в Wireshark

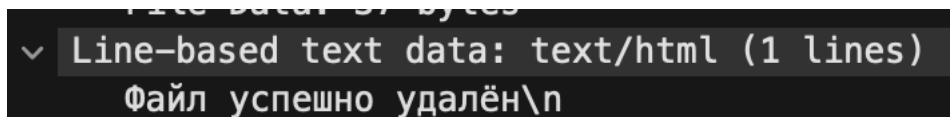


Рисунок 19 – Тело DELETE-ответа в Wireshark

```
127.0.0.1 -- [26/Apr/2026 16:39:50] "DELETE /my_first_file.txt HTTP/1.1" 200 -
127.0.0.1 -- [26/Apr/2026 16:39:50] "DELETE /my_first_file.txt HTTP/1.1" 200 -
```

Рисунок 20 – Результат в терминале

6) Попробуем получить информацию и скачать созданный файл с помощью браузера. Обычный ввод адреса в строку браузера автоматически инициирует GET-запрос

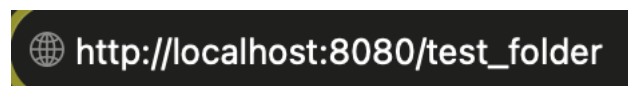
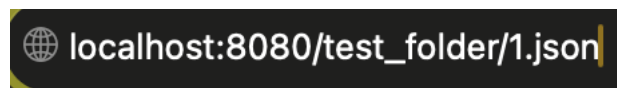


Рисунок 21 – Адресная строка в браузере (каталог)

```
[  
  ".DS_Store",  
  "1 copy.json",  
  "1.json",  
  "1 copy 2.json"  
]
```

Рисунок 22 – Получение информации о каталоге через браузер



localhost:8080/test_folder/1.json

Рисунок 23 – Адресная строка в браузере (файл)

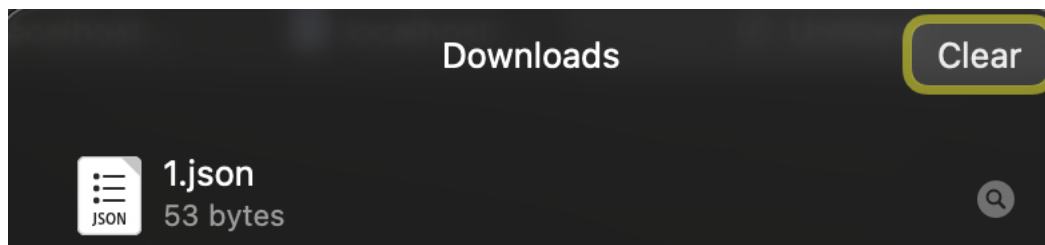


Рисунок 24 – Скачивание файла через браузер

HTTP	531	GET	/test_folder/1.json	HTTP/1.1
HTTP/JSON	109	HTTP/1.1	200 OK , JSON (application/json)	

Рисунок 25 – Перехват скачивания в Wireshark

Вывод: в ходе выполнения лабораторной работы было успешно разработано удаленное файловое хранилище, предоставляющее REST API по протоколу HTTP. Практическая реализация на языке Python с использованием микрофреймворка Flask позволила глубоко изучить принципы работы с HTTP-методами (GET, PUT, DELETE, HEAD) и маршрутизацией. Была реализована корректная работа с файловой системой операционной системы macOS (сохранение, удаление, чтение метаданных файлов и директорий). Работоспособность созданного API успешно подтверждена тестированием через утилиту Postman и веб-браузер.